Altiora · 未来的我

数字孪生分身 + 代价测算引擎

技术文档

传智杯大赛 · Vibe Coding 挑战赛 · B 组

赛道 8：个人效能与数字孪生具身仿真

目录

章节	内容
1	作品概述
2	需求分析
3	架构设计
4	意图控制策略（重点）
5	核心功能与确定性逻辑
6	AI 协同与演进链路（重点）
7	工程质量与安全
8	测试方案与结果
9	可复现性论证
10	创新点小结
11	部署方案
12	不足与展望

1. 作品概述

1.1 一句话定位

市面上的 AI 助手、日历、习惯养成 App 都在替用户做决定。Altiora 不替你决定——它只把每个选择的代价算清楚，让你自己挑。核心是一个「数字孪生分身」：基于用户的日常数据（目标、软肋、睡眠硬约束、规矩），在浏览器内实时测算每个日程决策的代价。

1.2 时代背景与产业需求

2026 年，自然语言已成为新一代的编程语言。Vibe Coding 从极客调侃演变为企业快速验证 MVP 与敏捷开发的主流手段，代码编写成本趋近于零，定义问题、产品设计、系统架构编排以及指挥 AI 的能力成为核心竞争力。产业界对「超级个体（Solo Developer）」的需求，正是本作品的出发点——一个人 + AI，在几小时内交付一套完整可用、可复现的系统。

1.3 解决的痛点

- 现有助手替用户做选择，用户失去掌控感，完成率一掉就想「重开」。
- 日程冲突只有「哪件更重要」的粗暴排序，没有「代价」视角，用户无法预判损失。
- AI 生成内容不可信：模型会编时间、编数字、给鸡汤，用户无法信任排程结果。
- 档案变摆设：很多产品的「设置」参数并不真正影响后续行为，用户填了等于没填。

1.4 核心创新

代价账单代替记分板：冲突时给出 A/B/C 三个选项，每个选项的得与失（金钱、时间、睡眠、后悔值）都摆出来，不替用户选。
LLM 只做翻译，不做计算：所有时间差、金钱、冲突分钟数由本地确定性代码算出，LLM 只把自然语言翻成结构化字段、把结果说成人话。
档案不是摆设：精力高峰、睡眠硬约束、软肋、目标都真实影响排程——改一个参数，今日方案肉眼可见地变。

1.5 与赛题契合度

赛道 8「个人效能与数字孪生具身仿真」的官方说明原文为：「面向日程、笔记、日常，快速搭建一套精致效动的『数字孪生分身』等。支持自适应日程冲突调整。」拆开是三个要求：①覆盖日程/笔记/日常三类日常数据；②做出「分身」；③冲突能自适应调整。逐条对应如下。

官方要求	本作品的实现	可验证处
日程	日/周/月三视图时间轴、课表导入、事件增删改	时间轴页签；技术文档 5.1
笔记	生活碎碎念（记日程 / 记规矩 / 记一件小事），三层记忆保留	碎碎念页签；技术文档 5.3
日常	情绪识别与回应、主动关心（回访/未画勾/空档）	主页对话；技术文档 5.3
数字孪生分身	把用户可计算的部分建成参数：时间、精力、软肋、目标节奏	画像页签；技术文档 10.3
自适应日程冲突	冲突检测 + 10 分钟缓冲 + 代价账单 + 空档重排，全部本地确定性计算	技术文档 5.1 / 5.2

表 1-1 赛道要求与本作品逐条对应

- 关于「分身」的边界：本项目不生成第二个「你」去对话，只把可计算的部分参数化并驱动决策。这个取窄的定义写在 10.3，与赛道用词一致，但不做超出实现能力的承诺。
- Vibe Coding 范式：由 Solo Developer 指挥 AI 完成，全流程有 Prompt 快照、赛期内每次改动有细粒度 Git 提交，可按 9.5 的演进脚本复现。

2. 需求分析

2.1 目标用户

高校学生与职场新人——日程密集、目标模糊、容易被 AI 助手的「鸡汤」和「替你决定」带偏。他们需要的不是又一个待办清单，而是一个能把决策代价算清楚的参谋。

2.2 核心使用场景

场景	用户行为	系统响应
冲突决策	「周五下午3点家教和组会撞了」	给出 A/B/C 三选项及代价，让用户自己挑
目标规划	「我想考上研究生」	抽取为长期目标，写入档案，影响后续排程优先级
精力优化	改档案精力高峰为 14:00–17:00	今日方案把脑力活挪进高峰窗
久别重逢	3 天以上没打开	翻日记：「上次你说要考试，后来怎么样了？」
情绪接住	「今天没睡好」	一句话接住情绪，不出表格不灌鸡汤

表 2-1 核心使用场景

2.3 功能需求清单

按 P0/P1/P2 分层（来自迭代 2 的 AI 协同决策）：

- P0：日程输入、冲突检测、代价账单（A/B/C）、删除与重算
- P1：长期目标抽取与记忆、精力高峰驱动的今日方案排程、记忆三层（长期/聊天/日记）
- P2：语音输入（Vosk 离线）、语音输出（系统 TTS）、AI 在线模式（BYOK）

2.4 非功能需求

- 可复现：同一输入永远同一输出，排程逻辑不依赖 LLM。
- 数据安全：Key 只存浏览器，服务端不落盘；自填 URL 防内网探测。
- 离线可用：双击 HTML 即可用全部文字功能，不依赖网络。
- 优雅降级：无 Key 或 Key 失效时，所有功能依然完整，只是文案偏机械。

3. 架构设计

赛题要求阐述「AI 生成的代码采用了什么架构，你如何约束它」。本作品采用三段式混合驱动架构，核心约束是：第二段（计算器）不含任何概率成分，同一输入永远同一输出。这是用户能信任它的唯一理由，也是可复现性的根基。

3.1 三段式架构

阶段	职责	是否调用 LLM
① AI 翻译官	自然语言 → 结构化字段（意图/事件/记忆/目标/情绪）	是，失败自动退回本地规则
② 计算器	冲突检测、代价账单、今日方案排程	否，纯确定性代码
③ AI 表达	把情绪与计算结果说成人话	是，失败回退确定性文案

表 3-1 三段式架构

三段之间只通过结构化数据传递，LLM 永远不接触数字计算。计算器的输出（facts）是 AI 表达的唯一输入，且表达层有数字校验防止编造。

3.2 前后端职责划分

前端（单 HTML 文件）：所有状态、记忆、排程、冲突计算都在浏览器内完成。双击 HTML 即可用文字与 AI，不需要 Python。后端仅提供两个可选能力——语音识别（Vosk 离线模型）和可选的 LLM 代理。不存用户数据，不写 Key 日志。

3.3 数据流

用户输入 → LLM 翻译（directUnderstand）→ 结构化字段 → 本地计算器（detectConflicts / buildTodayPlan）→ 结果 facts → LLM 表达（directSay）→ 展示。整个链路中，LLM 永远不接触数字计算，数字只在计算器里产生，在表达层被校验。

3.4 关键设计原则

- LLM 边界写死在代码里：intent 不是 emotion/chat 时，模型生成的 reply 被直接丢弃。
- 只说开始时间怎么办：带时段词（上午/中午/下午/晚上/夜里）就先按 1 小时摆进时间轴，并在回执里明说「我先按 1 小时算」——不偷偷替用户定时长；不带时段词（「3点开会」）则先反问「凌晨还是下午」，用户回一个「下午」就地补全，不必重说整句。
- 「已经发生」的句子不当日程：如「我 3 点就醒了」「下午 2 点睡了一觉」，命中状态词（醒/睡/起床/到家/下班/迟到）或句首「就/才/已经/刚」时直接放弃建事件，宁可问一句。

- 双模式自动切换：无 Key 走本地规则，有 Key 走 AI 在线，功能都完整。
- BYOK（自带 Key）：Key 只存浏览器 localStorage，请求由浏览器直连厂商，服务端不落盘。
- 确定性的归确定性，生成的归生成：排程由本地规则算，LLM 只负责把结果说成人话；LLM 失败一律回退确定性文案。

3.5 AI 生成代码的约束策略

如何约束 AI 生成的代码不跑偏，是 Vibe Coding 的核心问题。本作品采用四层约束：

- 角色约束（SYSTEM Prompt）：从输入侧告诉模型只能做翻译，不能计算、不能决策。
- 字段约束（JSON Schema 思维）：LLM 输出必须是固定结构（intent/event/memory/goal/social/reply），缺字段或类型错直接判失败。
- 运行时校验：reply 丢弃、数字校验、_BANNED 正则，三层运行时拦截。
- 回退约束：任何校验失败都回退本地确定性逻辑，保证功能不中断。

4. 意图控制策略

这是本作品的核心，权重 20%。我们用三层锁把 LLM 锁在「翻译官」的角色里，防止它越界做计算、做决策、说鸡汤。每一层都有实战 Bug 佐证其必要性。

4.1 三层锁总览

层级	机制	作用	实现位置
第一层	SYSTEM Prompt 边界	从输入侧告诉模型能做什么、不能做什么	llm.py SYSTEM / 前端 LLM_SYS
第二层	_BANNED 正则	过滤客服腔和鸡汤句	llm.py _BANNED / 前端 LLM_BANNED
第三层	reply 丢弃 + 数字校验	schedule/safety 场景强制丢弃 reply；say 阶段数字必须来自	llm.py 数字校验 / 前端 directUnderstan

表 4-1 意图控制三层锁

4.2 第一层：SYSTEM Prompt 边界

SYSTEM prompt 明确列出 7 种意图（safety/emotion/schedule/memory/goal/social/chat），并对每种 intent 的 event/memory/goal/social 字段做了抽取规则。最关键的是「绝对不能做」清单：

```
【绝对不能做】
- 不计算时间重叠、时长、金钱、代价
- 不替用户选方案，不说「你应该选 A」
- 不编造用户没说的地点、时间、人物
- 不说「加油」「你可以的」「相信自己」，不喊口号
- 不复述用户原话，不用客服腔
```

reply 的写法也被严格约束：intent=schedule 且 event 完整时 reply 必须为 null（账还没算，不能抢答）；intent=safety 时 reply 为 null（前端有固定安全话术）。

4.3 第二层：_BANNED 正则过滤

即使 SYSTEM prompt 写了禁止，模型仍可能输出客服腔。所以在代码层加了一道正则，命中即作废。前后端各有一份完全相同的正则，因为 LLM 调用有两条路径：浏览器直连（directUnderstand）和服务端代理（understand），两条路径都要过滤。

```
_BANNED = re.compile(
    r"嗯[，,]\s*我在听|我在听|你接着说|我在这儿|我都在|"
    r"会好起来的|会好一点|加油|你可以的|相信自己|"
    r"慢慢来|不要放弃|你能做到"
)
# 命中 → reply = None，前端用本地兜底话术
```

4.4 第三层：reply 丢弃 + 数字校验

4.4.1 schedule/safety 场景强制丢弃 reply

完整日程的账必须由计算器算完再说，LLM 不能抢答。安全场景由前端固定话术处理，不让模型自由发挥。

```
# 服务端 llm.py
if intent in ("safety",) or (intent == "schedule" and event):
    reply = None

# 前端 directUnderstand（同样逻辑，双保险）
if intent == "safety" or (intent == "schedule" and event):
    reply = None
```

4.4.2 say 阶段数字校验

第二段翻译官（directSay / say）把计算器的结果说成人话。为防止模型编造数字，我们校验：输出中出现的所有数字，必须是 facts 里已有的数字，多一个就整句作废。

```
# 前端 directSay
const allowed = new Set((JSON.stringify(facts).match(/\d+(?:\.\d+)?/g) || []));
const nums = text.match(/\d+(?:\.\d+)?/g) || [];
if (nums.some(n => !allowed.has(n))) return ""; // 编造数字 → 作废
```

服务端 say() 同样实现了数字集合校验（_numbers 函数），双路径一致。

4.5 失败回退机制

任何一层校验失败，都不会让用户看到空白或错误——而是回退到本地确定性文案。

- directUnderstand 返回 null → 前端用 parseEventsFromText 本地解析，或追问时间。
- directSay 返回空 → 前端用拼好的确定性文案（如「『考研复习 14:00-16:00』加进时间轴了，没冲突」）。
- explainPlan 失败 → 用 planCaption() 拼一句说清「为什么这么排」。

核心原则：LLM 是锦上添花，不是功能前提。没有 Key 或 Key 失效，所有功能依然完整可用。

4.6 实战 Bug 佐证：「LLM 忽略档案」

这是三层锁设计的直接动因。现象：模型拿到「昨日方案」后，输出一句通用套话，完全没提精力高峰时段，也没提档案里的睡眠底线/软肋——档案变摆设。

根因

- Prompt 只说了「解释方案」，没强制要求引用档案字段；

- 输出没有任何运行时校验，说漏了也直接展示。

修复（双保险）

- Prompt 层：把「必须明确说出哪件事被排进了哪个精力高峰时段」写成铁律；
- 代码层（explainPlan）：精力高峰一旦填写，校验输出必须包含时段字符串或被挪事项名，任一不满足 → 丢弃 LLM 输出 → 回退 planCaption()。

回归验证

```
LLM 输出「我把考研复习排到 14:00–17:00，因为你这段精神最好」
  → 命中时段「14:00–17:00」 → 通过
LLM 输出「今天排得很满，你自己注意休息」
  → 未命中 → 回退确定性文案
```

这个 Bug 的完整调试过程记录在 PROMPT_LOG.md 与 AI协同对话历史.md 中。

4.7 Prompt 上下文设计详解

比赛要求阐述「如何设计上下文和 Prompt」。本作品的上下文设计遵循「只给事实，不给结论」原则——把档案和输入的事实结构化地喂给 LLM，让它做翻译，不做推理。

4.7.1 directUnderstand 的报文构造

意图识别阶段，user 报文由两部分组成：档案摘要 + 用户原话。档案摘要 profileBrief 只包含计算器需要的字段（目标、软肋、睡眠硬约束、精力高峰、规矩），不包含冗余信息，避免模型分心。

```
# user 报文结构
档案：
{profileBrief(DATA.profile)}

用户原话：
{text}

请按 SYSTEM 规则抽取结构化字段。
```

设计要点：档案在前、原话在后，让模型先建立「用户是谁」的上下文，再理解当前输入。字段清单与 SYSTEM 中定义的 intent 类型一一对应。

4.7.2 explainPlan 的报文构造

今日方案说明阶段，user 报文包含档案摘要 + 方案事实 planFacts（JSON）。planFacts 只包含计算器已算出的事实（哪些事被挪、挪到哪、冲突情况、睡眠底线是否保住），LLM 只需把这些事实说成一两句话。

```
# user 报文结构
档案：
{profileBrief(DATA.profile)}

方案（只解释，不改）：
{JSON.stringify(planFacts)}
```

设计要点：planFacts 是 JSON，数字已确定，LLM 无法编造（4.4.2 数字校验兜底）。「只解释，不改」明确告诉模型这是翻译任务，不是决策任务。

4.7.3 directSay 的报文构造

情绪与结果表达阶段，user 报文包含 intent + 结构化 facts。模型根据 intent 选择语气（emotion 共情、schedule 中性陈述），facts 中的数字是唯一允许出现的数字。

4.8 完整 SYSTEM Prompt（节选）

以下为前端 LLM_SYS 的核心节选（V4 版本），展示意图分类、字段抽取与多意图约定。完整版本见 Altiora-未来的我.html 的 LLM_SYS 常量，后端 llm.py 的 SYSTEM 与之一致。

```
你是 Altiora，一个只做「自然语言 → 结构化字段」翻译的助手。

【意图分类】只能输出以下 7 种 intent 之一：
safety / emotion / schedule / memory / goal / social / chat

【抽取规则】
- 中文数字（三点）、相对日期（下周三）、口语时长（复习两小时）
  都由你归一化成 24 小时制 HH:MM，本地规则不参与在线路径。
- schedule 的时间给在 events 数组里，每项含 name/start/end，最多 4 件；
  可附 date（YYYY-MM-DD），拿不准就不给。
- 一句话里可能同时有好几层意思：情绪、目标、软肋可以一起给。
  weakness 只在用户描述「反复发生的自身规律」时给，20 字以内，
  只说一次的心情不算。不许下心理诊断。

【绝对不能做】
- 不计算时间重叠、时长、金钱、代价
- 不替用户选方案
- 不编造用户没说的地点、时间、人物
- 不说鸡汤、不喊口号

【reply 规则】
- intent=schedule 且 events 至少一件完整 → reply 必须为 null（账由计算器算完再说）
- intent=safety → reply 必须为 null
- 其他 intent 可写一句简短人话，不超过 40 字

输出 JSON：
{"intent":"...","events":[...],"emotion":"...","weakness":null,
 "goal":null,"goal_horizon":null,"reply":null}
```

注意 V4 的两处变化：schedule 由单件 event 改为 events 数组（一句话多件事）；goal 与 weakness 不再受主意图限制——用户说「我想考研，可一焦虑就刷手机」时，情绪、目标、软肋会同时抽取，而不是只保留一个 intent。

5. 核心功能与确定性逻辑

5.1 冲突检测与代价账单

detectConflicts 遍历同一天的事件两两比较，重叠分钟数 > 0 即为冲突。makeConflict 为每对冲突生成 A/B/C 三个选项：

选项	含义	代价示例
A	去优先级高的那件	损失家教 2 小时 × 80 元 = 160 元
B	去优先级低的那件	放弃重要的事，事后可能后悔
C	把低优先级的顺延	连轴转，可能破睡眠硬约束

表 5-1 冲突代价账单三选项

detectConflicts 核心逻辑

遍历同一天事件两两比较，计算重叠分钟数。重叠 > 0 即冲突。核心是时间区间重叠判定：

```
# 两事件 [s1,e1] 与 [s2,e2] 重叠判定
overlap = min(e1, e2) - max(s1, s2)
if overlap > 0:
    conflict = makeConflict(a, b, overlap)
# makeConflict 按优先级决定 hi/lo：
#   A = 去 hi，代价 = lo 的 wage × dur(lo)
#   B = 去 lo，代价 = 定性「放弃重要的事」
#   C = lo 顺延到 hi.end + 30min，检查是否破睡眠底线
```

注意：makeConflict 里的金钱计算（wage × dur）、优先级判断、顺延时间（hi.end + 30 分钟）全部是本地计算，LLM 完全不参与。这是「LLM 只做翻译」原则的直接体现。

5.2 今日方案排程（buildTodayPlan）

这是「档案不是摆设」的核心证明。规则：

- 课程/会议/睡眠 = 固定项，原地不动；学习/兼职 = 可动项。
- 可动项按「优先级小者先 → 时长长的先」排序。
- 在精力高峰时间窗内贪心塞入，撞到固定项就绕过，塞不下保持原位。

改档案里的 preferred_focus_hours，方案肉眼可见地变：

```
档案精力高峰 = "14:00–17:00"
  →「考研复习」从 15:00-17:00 挪到 14:00-16:00（moved=true）

档案精力高峰 = "08:00–11:00"（此时 08:30-10:10 有课占窗）
  →「考研复习」保持原位（moved=false）

档案精力高峰 = 未填
  → 方案按原时间排，不优化
```

focusWindowOf 与贪心塞入逻辑

排程算法确定性的核心：先求可塞入的时间窗，再按固定顺序贪心塞入。

```
# focusWindowOf：从 preferred_focus_hours 中扣除固定项占用，
#   得到剩余可塞入的连续子窗
window = peak_hours
for fixed in fixed_events:
    window = window - fixed   # 区间减法

# buildTodayPlan：按「优先级升序 → 时长降序」排序可动项，
#   从窗头开始塞，撞到固定项就跳过，塞不下保持原位
for task in sorted(movable, key=lambda t: (t.priority, -t.dur)):
    if fits(window_head, task.dur):
        place(task, window_head)
        window_head += task.dur
    else:
        keep_original(task)   # 不动，不报错
```

算法复杂度 O(n log n)（排序主导），无递归、无随机、无 LLM 调用。同一组事件 + 同一档案永远产出同一方案。

5.3 记忆三层

层	存什么	保留多久	谁用
长期记忆 (profile)	目标、软肋、睡眠硬约束、规矩	一直	计算器输入，只有明说才写
聊天记忆	最近 3 轮对话原文	超 6 条滚掉	只接半句（「就那个」「三点」）
日记	每行「日期：摘要」	最多 180 行	隔 3 天以上回来才翻

表 5-2 记忆三层设计

三层各管各的，用「保留时长」划边界，避免记多了像监视、记少了每句从零开始。日记按「目标 → 排了什么 → 那会儿的状态」拼摘要，不存原话。

5.4 目标抽取：在线交给模型，离线保留七步兜底

在在线路径（有 Key / server.py）下，目标由模型直接从原话抽取，前端不再跑正则；七步规则只保留给无 Key、断网或模型返回坏 JSON 的离线路径。用户说「你好啊，我想当科学家」，存进去的应该是「成为科学家」而不是原话。七步规则如下：

- ① 剥寒暄（去掉问候、语气词）
- ② 锚点词截断（我想/我要/我希望/我的目标是/我以后想 等）
- ③ 去时间/连接词（明天、以后、而且）
- ④ 动词归一（当/做 → 成为）
- ⑤ 去语气词（啊、呢、吧）
- ⑥ 长度护栏（过长则截断）
- ⑦ 情绪/当天事挡在门外（我想哭、我明天想去图书馆 → 不存）

修复前曾出现「我以后想成为科学家」解析失败的 Bug——M 正则缺「我以后想」，而 goalHorizon 判定长期的关键词里却有「以后想」，两头不一致。修复后两者关键词完全对齐。这次修复也暴露了规则模板的局限：靠穷举关键词永远有漏网句式，因此从 V4 起，在线路径不再依赖这套正则。

5.5 多意图抽取与三道护栏

用户说话很少只有一个意思。最常见的一句是：「我最近想考研，可一焦虑就刷手机，今天又刷到两点，好烦。」V1–V3 只能命中一个意图（情绪），目标和软肋都会丢。V4 把协议改成「主意图 + 附加层」，三个字段可以同时产出。

字段	谁产出	护栏（本地）
intent / emotion	模型判定	LLM_INTENTS / LLM_EMOTIONS 白名单，越界置 none
goal + goal_horizon	任意意图下均可给	GOAL_HORIZONS 白名单，长度 ≤ 60，去前缀
weakness	从自我描述中提炼	cleanWeakness 限长 24、WEAK_BANNED 禁建议/鸡汤、去重且上限 4
events	模型抽取	cleanEvent 校验名非空、HH:MM、结束晚于开始

表 5-3 多意图字段与本地护栏

护栏的作用不是限制理解，而是把「理解」和「评判」分开：模型可以判断「一焦虑就刷手机」是一条规律，但不能把它写成建议或诊断。命中「建议 / 应该 / 加油 / 相信自己」的软肋一律丢弃。

前端会把本次抽取结果折叠成一张「模型理解」卡片附在回复下方，用户点开即可核对意图、情绪、目标、软肋与事件，理解错了能立刻看见——这是可解释性，也是答辩时最直观的证据。

5.6 产品精细度：状态可见、主动关心、成长可见

这一节对应「用户体验与产品精细度」维度。我们只做四件能被看见、且不撒谎的事：

- 状态可见：等待时不是一句笼统的「正在思考」，而是三个阶段条——理解这句话 › 落进日程算代价 › 组织回话。阶段按真实进度推进；走本地规则时直接标「本地规则」，不假装那头有个模型。占位气泡不进聊天记忆、不被朗读。
- 主动关心有由头：只在四种情况下开口——超过 3 天没来（翻日记回访）、当天有排好的事没画勾、近期有大段空档、中长期目标临近截止（如「离『六级』还有 35 天」）。没有由头就不打扰，避免变成又一个推送器。
- 成长可见，但不打分：画像页的「这 30 天」用 30 个格子只摆事实（回来过几天、画了几个勾、排了几场），不折算成分数；「你反复说的话」完全在本地用两字滑窗统计词频，不经过模型，也不做情绪判断。
- 用户能纠错，空白有引导：画像里记错的软肋点一下就删（「以后不拿它说事了」），不必忍受一个错标签；碎碎念页没有记录时给三种可直接照说的句子（进时间轴 / 记成规矩 / 记成小事），30 天格子全空时说明「用几天会一天天填起来」，而不是让用户面对一片灰格猜。

这四处都不增加新的能力，只把已有的确定性结果如实展示出来——用户体验分不靠动画堆，而靠「它说的和它做的一致」。

6. AI 协同与演进链路

本章节对应两个评分维度：「解题思路与演进链路」（25%）与「方法创新与意图控制」（20%）。记录与 AI 协作的真实过程，证明作品不是一次性拼接，而是逐步迭代演进。完整对话历史见 AI协同对话历史.md 与 prompt-snapshots/。

6.1 协同方法论

与 AI 协作的核心原则是：我负责定义问题与验证，AI 负责产出与建议。每一轮遵循「我提 Prompt → AI 给方案 → 我判断并要求实测 → 验证通过才落地」。AI 的产出不直接合入，必须经过本地断言或浏览器实测验证。

混合驱动架构的选型决策也是与 AI 对话得出：纯 LLM 不可复现、纯规则生硬，采用「确定性排程 + LLM 装饰」分离，互不污染。

6.2 六个核心迭代

迭代	主题	关键决策
1	目标规划 + 自适应重规划	闭环：目标→计划→执行→状态→重规划；重规划由本地确定性逻辑驱动
2	功能分层 + 记忆库选型	P0/P1/P2 分层；放弃复杂数据库，用 localStorage 轻量记忆
3	产品表现形式	单 HTML 原型先行，左输入右时间轴，2 分 25 秒出可交互原型

迭代	主题	关键决策
4	LLM 配置面板 + 语音	BYOK（Key 只存浏览器）+ DNS 重绑定防护；系统 TTS 免费离线
5	修 Bug：目标存成整句	extractGoal 拆检测与抽取两步，七步规则，7 正例 + 2 反例全过
6	记忆分三层	长期/聊天/日记，保留时长划边界，3 天以上翻日记回访

表 6-1 六个核心迭代（完整 Prompt 链见 AI协同对话历史.md）

每轮迭代的完整 Prompt 原文、AI 输出、决策与截图索引，均记录在 AI协同对话历史.md 中，覆盖远超比赛要求的「≥3 个核心功能迭代完整 Prompt 链」。以下选取迭代 5 与迭代 6 展开，展示「我提 Prompt → AI 给方案 → 我要求实测 → 验证通过才落地」的完整协作链路。

6.2.1 迭代 5 详解：目标抽取七步规则

我的 Prompt：「有个 bug，我输入『你好啊，我想当科学家』，直接把原话放进长期目标了。只留目标本身：上面那句最后存的应该是『成为科学家』，不是原话。语言逻辑是什么样子的」
AI 输出：把「检测」和「抽取」拆成两步，重写 extractGoal()——剥寒暄 → 锚点词截断 → 去时间/连接词 → 动词归一 → 去语气词 → 长度护栏 → 情绪/当天事挡在门外。
实测对照表（7 正例 + 2 反例）：

输入	抽取结果	说明
我想当科学家	成为科学家	当→成为
我想成为科学家	成为科学家	已是成为
我想做老师	成为老师	做→成为
我的目标是考上研究生	考上研究生	判断词剥离
我希望减肥成功	减肥成功	希望类锚点
我打算学编程	学编程	打算类锚点
我以后想成为科学家	成为科学家	以后想锚点 + 去时间词
我想哭	（不存）	情绪，挡在门外
我明天想去图书馆	（不存）	当天事，挡在门外

表 6-2 extractGoal 实测对照（9 例全过）

决策：9 例全过才提交。这一步把「检测关键词」和「抽取内容」彻底分离，避免了「检测用 A 词、存进去用原话」的 Bug。

6.2.2 迭代 6 详解：记忆三层

我的 Prompt：「做记忆分层，分成长期记忆和聊天记忆和日记。长期记忆要符合软件优势，聊天记忆短一点主要是上下文不太突兀。日记主要是每次聊完存一行『日期：摘要』，平时不掺和，超过 3 天没来，翻日记上次你说要考试，后来怎么样了」
AI 设计判断：记多了像监视，记少了每句从零开始。三层用「保留时长」划边界，各管各的。
实测数据：

输入：「我要考试」→ 追问时间 →「明天下午三点到五点」

→ 补成事件 15:00-17:00 √
聊完 5 轮后：聊天记忆稳定 6 条 √
日记落成：10月24日：想成为科学家，排了考试 15:00-17:00，那会儿很累 √
把日期调到 4 天前重开：
开场即「上次（10月20日）你说想成为科学家，后来怎么样了？」 √

决策：日记不存原话，只存「目标→排了什么→状态」摘要，既保护隐私又能在久别重逢时唤起上下文。聊天记忆只接半句（「就那个」「三点」），不参与长期决策。

6.3 实战 Bug 修复合集

以下为提交前打磨阶段与 AI 协作排查的 5 个真实 Bug，每条均含「现象 → 根因 → 修复 → 回归」。

Bug A：规矩带前导逗号

- 现象：「记住，我晚上容易刷手机」→ 画像显示 `，我晚上容易刷手机`。
- 根因：本地规矩提取截断后未清理首尾标点。
- 修复：新增 stripPunct() 去首尾 CJK/ASCII 标点，在 cleanMemory() 与两条本地提取路径套用。
- 回归：画像页不再出现前导标点。

Bug B：「我以后想成为科学家」解析失败

- 现象：extractGoal 返回空，去掉「以后」就正常；讽刺的是 goalHorizon 里「以后想」是判定 long 的关键词，提取函数却先挂了。
- 根因：extractGoal 的 M 正则缺「我以后想/以后想」（因为「我想」要求紧邻），提取与判定关键词不一致。
- 修复：M 正则补「我以后想/以后想/我以后要/以后要」；goalHorizon 同步补「以后要」。
- 回归：返回「成为科学家」/「long」。

Bug C：日记层看不见（5 分钟才 flush）

- 现象：设完目标后日记页要等 5 分钟空闲才落一条，演示时摸不到「久别重逢」卖点。
- 根因：sessFlush() 只在 idle 定时器触发，无主动触发点。
- 修复：sessFlush() 在设目标（LLM + 本地）和 addStructuredEvent() 后立即调用；空闲阈值 5min→60s；日记页渲染自动日记 + 久别重逢预览。
- 回归：设完目标立即落日记。

Bug D：LLM 缺字段导致整页崩溃

- 现象：LLM 返回缺 start/end 的事件，或 localStorage 加载损坏数据时，toMin(undefined) 抛 TypeError，整页白屏。
- 根因：toMin、addStructuredEvent、日记渲染对脏数据无守卫。
- 修复：toMin 加 typeof 检查 + includes(":") 守卫；addStructuredEvent 设默认值；加载时 filter 校验；日记 e.s.split() 加存在性判断。
- 回归：脏数据不再崩溃。

Bug E：extractGoal 前导冒号 + 判断词「是」残留

- 现象：「我想：成为科学家」→ `：成为科学家`；「我的目标是：成为科学家」→ `是：成为科学家`。
- 根因：与 Bug A 同病根——截断后标点没刮干净，且判断词「是」未剥离。
- 修复：判断词「是」加进剥离列表，归一后统一 stripPunct。
- 回归：9 个用例全过，正常 case 结果不变。

6.4 提交与快照对照

评委抽查时可将 Git 提交与 Prompt 快照逐一对号。本仓库共 19 次提交，均为 2026-11-14 建库后的真实改动，可逐条 git show；六个核心迭代的原始开发提交未随仓库迁移，其过程证据为 prompt-snapshots/ 的 11 张真实对话截图（见 9.5）。下表为赛期内代表性对照：

改动	代表提交	佐证位置
服务端限流 + BACKEND 路径修复	05a5cb2	7.6 节
.env 行内注释解析 / AI 超时 9s→20s	6e9c928	7.6 节
情绪引用原话 / 目标倒计时 / 只说起点	44c1f39	5.3、5.4 节
「3点开会」歧义先反问	a8f2c61	4.5 节
目标名不再吞掉日期	eb383bd	AI协同对话历史 第 119 行
三步状态条（本地规则如实标注）	4b83e97	5.6 节
缺点可删 / 空状态引导	e721f2f	5.6 节
favicon 204 / 线上验收	c222ba0	11.5 节
演进脚本与复现性声明	8fb982a	9.5 节、docs/replay.md

表 6-2 本仓库真实提交与证据对照（共 19 次，此为代表性节选）

7. 工程质量与安全

7.1 DNS 重绑定防护

用户自填的 LLM Base URL 可能被用来探测内网。我们做了两层防护：

- 第一层：字符串前缀过滤，挡掉 localhost、127.x、10.x、192.168.x 等。
- 第二层：DNS 解析后逐一检查返回 IP，若落在私网/回环/保留范围则拒绝（防 DNS 重绑定）。

```
def _safe_base(base):
    u = urlparse(base)
    if u.scheme != "https" or not u.hostname: return False
    # 第一层：字符串前缀
    if any(h == p or h.startswith(p) for p in _PRIVATE_HOSTS): return False
    # 第二层：DNS 解析后查 IP
    infos = socket.getaddrinfo(h, u.port or 443, type=socket.SOCK_STREAM)
    for info in infos:
        if _is_private_ip(info[4][0]): return False
    return True
```

7.2 BYOK 安全设计

- Key 只存浏览器 localStorage，服务端不落盘、不写日志、不进仓库。
- 请求由浏览器直连厂商 API，不经过服务端中转。
- 服务端 .env 模式仅用于单人自用；开放给他人时强制 BYOK。

7.3 输入校验

- event 必须有 name + 合法 start/end（HH:MM），否则丢弃。
- 时间区间必须 end > start，否则 event = null。
- 所有字段有长度上限（name ≤20，memory ≤60，goal ≤60）。
- extra_headers 做头注入防护：过滤 Host/Content-Length 等危险头，限制长度。

7.4 单文件可维护性

前端为单 HTML 文件，便于部署与评审。为避免「AI 生成的单文件不可维护」问题，代码按功能分区（意图识别/计算器/记忆/表达/渲染），三区通过结构化数据耦合，LLM 相关代码集中在 directUnderstand/directSay/explainPlan 三个函数，修改 LLM 行为不会影响确定性计算。

7.5 性能与无死循环论证

评审维度要求「无严重安全漏洞与性能死循环」。本作品所有计算路径均有明确上界：

- 冲突检测：两两比较 $O(n^2)$，n 为单日事件数（通常 <20），常数级耗时。
- 排程贪心：排序 O(n log n) + 线性塞入 O(n)，单次生成今日方案 <1ms。
- 记忆滚窗：聊天记忆超 6 条即 shift 掉最早一条，数组长度恒定；日记最多 180 行，超限截断。
- 无递归：所有循环均为有界 for/while，不存在递归调用栈溢出风险。
- LLM 超时：directUnderstand/directSay 均设 fetch 超时（默认 15s），超时即回退确定性文案，不会挂起界面。

7.6 公网服务部署与调用限流

评审版以 同源部署方式提供：前端页面与 /api/understand、/api/say 由同一台服务器（同一个 server.py）提供，页面里用相对路径调用接口，因此不存在 https 页面调 http 接口的混合内容拦截，也不需要额外的反向代理或隧道。

服务端注入 Key 后，浏览器侧零配置即为「AI 在线」。为防止公网被刷爆免费额度，接口层加了一道进程内每日限流：

- 每 IP 每日 50 次（RATE_PER_IP_DAY），按天自动清零，互不影响。
- 全局每日 1000 次（RATE_GLOBAL_DAY），防止换 IP 绕开单 IP 限制。
- 只拦两个花额度的接口：/api/understand 与 /api/say；/api/asr 是本地语音识别，不计费，不限。
- 超限返回 429，前端 askLLM 收到非 2xx 即返回 null，静默回退本地规则，用户侧功能不降级、不报错。

客户端真实 IP 优先取 X-Forwarded-For 首段，兼容将来放在反向代理之后；没有该头时退回直连地址。计数器由一把线程锁保护，避免并发下重复计数。

```
RATE_PER_IP_DAY = int(os.environ.get("RATE_PER_IP_DAY", "50"))
RATE_GLOBAL_DAY = int(os.environ.get("RATE_GLOBAL_DAY", "1000"))

def _check_rate(request):
    today = date.today().isoformat()
    ip = _client_ip(request)
    with _rate_lock:
        if _rate["day"] != today:        # 跨天自动清零
            _rate["day"], _rate["ip"], _rate["total"] = today, {}, 0
        if _rate["total"] >= RATE_GLOBAL_DAY:
            return JSONResponse({"ok": False, "reason": "rate_limited_global"}, status_code=429)
        if _rate["ip"].get(ip, 0) >= RATE_PER_IP_DAY:
            return JSONResponse({"ok": False, "reason": "rate_limited"}, status_code=429)
        _rate["ip"][ip] = _rate["ip"].get(ip, 0) + 1
        _rate["total"] += 1
    return None
```

另一处部署级细节：.env 解析支持行内注释（KEY=50　# 说明）。最初的实现只做 strip，导致照着模板部署时 int("50　# 说明") 直接抛 ValueError、服务起不来；现按「# 前必须是空白」的规则剥离注释，既修掉崩溃，又不会误伤值里合法的 # 字符。

8. 测试方案与结果

#	操作	预期结果	实际结果
1	说「今天没睡好」	AI 用一句话接住情绪，不出表格	通过
2	「我明天下午要和导师讨论」	听懂意图但时间不全 → 追问，不编时间	通过
3	「周五下午3点到5点去家教」	抽成事件排进时间轴，算出冲突与代价	通过
4	「记住，我晚上容易刷手机」	写入画像，冲突时代价栏出现	通过
5	改精力高峰为 14:00–17:00，生成今日计划	可动的考研被挪进高峰窗	通过
6	配自己的 LLM Key	AI 在线模式，理解更准	通过
7	删除日程	hover 出 ×，点击删除，冲突重算	通过
8	「我以后想成为科学家」	目标存为「成为科学家」，长期	通过
9	3 天以上未打开	开场翻日记回访	通过

表 8-1 核心链路测试用例与结果

8.1 确定性断言测试

排程核心（focusWindowOf / buildTodayPlan / planCaption）脱离浏览器环境单独断言，同一输入永远同一输出：

```
精力高峰 = "14:00–17:00" → 考研复习挪到 14:00-16:00  √
精力高峰 = "08:00–11:00"（被课占）→ 不动              √
精力高峰 = 未填 → 不优化                              √
ALL PASS √
```

8.2 意图控制回归测试

extractGoal 抽取 9 个用例全过，含正例与反例；explainPlan 数字校验对编造数字的输出正确作废；_BANNED 对客服腔正确拦截。

8.3 意图控制三层锁专项测试

层级	测试输入	预期行为	结果
第一层 SYSTEM	「帮我算一下这两件事哪个更重要」	intent=schedule，reply=null，不替用户选	通过
第二层 _BANNED	用户情绪低，LLM 输出「加油你可以的」	命中 _BANNED → reply 作废 → 回退	通过
第三层 reply 丢弃	「明天下午3点到5点家教」	event 完整 → reply 强制 null，等计算器	通过
第三层 数字校验	facts 含 14:00，LLM 输出「15:00」	15 不在 facts 数字集合 → 整句作废	通过
第三层 数字校验	facts 含 160 元，LLM 输出「160 元」	160 在 facts → 通过	通过
回退机制	LLM 超时无响应	directSay 返回空 → 用 planCaption 兜底	通过

表 8-2 意图控制三层锁专项测试

9. 可复现性论证

官方评审维度中「过程可复现性」与「解题思路与演进链路」各占 25%，合计一半分值。提交要求逐条对照如下，每一条都能在仓库里点到。

官方提交要求	本作品的交付物	在哪看
Public 代码仓库	GitHub 公开仓库，含完整历史与分支配注	README 首屏链接
细粒度 Git 提交（禁止一次提交全量）	按功能/修复拆分的提交序列，含本次赛期内的变更	9.1 节
AI 对话历史导出或截图	11 张 Prompt 快照 + 逐条协同记录（含 3 次 AI 判断错误）	prompt-snapshots/、AI协同对话历
≥3 条完整 Prompt 链（你的提问、AI 建议、如何被你修正）	开发过程中的关键决策记录	AI协同对话历史.md
README：10 分钟极速复现（环境、依赖、变量模板）	第 3 节，含依赖清单与 .env 模板	README.md
技术文档 PDF ≤30 页	本文件，22 页	Altiora-技术文档.pdf
视频 5-8 分钟 MP4 1080p，含 ≥2 分钟 AI 协同过程实录	拍摄素材	—

表 9-1 官方提交要求与本作品交付物对照

以下从四个维度展开论证。

9.1 细粒度 Git 提交历史

赛题禁止「initial commit 包含全部功能」。本仓库建立于 2026-11-14，此前的迭代提交未随仓库迁移（说明见 9.5），因此下表只列本仓库真实存在、可 git show 逐条核对的提交；早期六个迭代以 prompt-snapshots/ 的 11 张真实对话截图与 AI协同对话历史.md 为过程证据——赛题明确允许以截图或导出替代工具内历史。

提交哈希	说明
8fb982a	docs: 新增演进脚本 docs/replay.md（六迭代 + 三 Bug 四段式）；README 补复现成功三标志
ce0809a	docs: README 增复现性声明五条；协同历史顶部补三条完整 Prompt 链索引
445d06d	docs: 技术文档新增 11.5 线上验收记录；5.6 补纠错与空状态引导
c222ba0	chore(server): /favicon.ico 返回 204，消掉控制台 404
e721f2f	polish(ux): 碎碎念空状态给说法、缺点可一键删除、截止日期说明随填写收起
4b83e97	feat(ux): 思考状态改真实三步，本地规则如实标注
eb383bd	fix(goal): 目标名别把逗号后的日期/考试安排一起吞进去
a8f2c61	fix: 「3点开会」先反问凌晨还是下午，回一个「下午」就地补全
44c1f39	feat: 情绪回话引用用户原话；主动关心带目标倒计时；支持只说开始时间
6e9c928	fix: .env 行内注释解析；服务端 AI 超时 9s→20s
05a5cb2	feat: server.py 每 IP + 全局每日限流，修复 BACKEND 路径

表 9-2 赛期内细粒度提交示例（完整历史见 git log）

9.2 Prompt 历史快照

prompt-snapshots/ 目录保存了 6 个核心功能迭代的完整对话链截图，每个都包含「用户提问 → AI 输出 → 决策/实测」。AI协同对话历史.md 提供可检索的文字版。远超比赛要求的「≥3 个核心功能迭代完整 Prompt 链」。

截图文件	迭代	内容
iter1-01-goal-discussion.png	1	完整对话链，用户先开口
iter1-02-goal-loop-design.png	1	AI 提出闭环，用户追问
iter1-03-adaptive-replan.png	1	确定「自适应」是赛题关键词
iter2-01-layered-design.png	2	按「自适应程度」分层功能
iter2-02-memory-vs-framework.png	2	为什么不搞复杂数据库
iter3-01-form-explained.png	3	通俗版产品描述
iter3-02-final-preview.png	3	2 分 25 秒出可交互原型
iter4-01-llm-config-panel.png	4	配置面板字段与安全验证
iter4-02-local-tts.png	4	确定用系统自带 TTS
iter5-01-goal-extraction.png	5	七步抽取规则与实测对照表
iter6-01-memory-layers.png	6	三层记忆表格与实测结果

表 9-3 Prompt 快照清单（共 11 张，覆盖 6 个迭代）

9.3 自检脚本

自检.py 提供十一项检查：Python 版本、四个依赖、语音模型完整性、前端关键控件、页面无真实 API Key、前端脚本语法、服务启动、识别接口连通。不联网、不调大模型、不消耗额度。全过退出码 0。

9.4 确定性断言

排程核心可脱离浏览器环境单独断言。同一输入永远同一输出，证明排程逻辑完全确定、可复现。详见 8.1 节。

9.5 演进脚本：照着走就能复现

赛题要求「通过 Prompt 记录和演进脚本进行确定性复现」。仓库 docs/replay.md 把六个迭代与三条关键 Bug 写成可执行步骤，每条固定四段：我提了什么 → AI 给了什么建议 → 我怎么判断与引导 → 怎么验证它真的修好了，并给出可复跑的输入与期望输出。

复现入口只有三步：git clone → python3 自检.py（退出码 0）→ python3 server.py（或直接双击 HTML）。随后用三个成功标志判断环境是否正常：右上角显示「AI 在线」或「本地规则」；发「今天有点累」得到一句情绪接住且不弹表格；发「周五下午3点开会」与「周五下午3点半做家教，两小时」分别得到时间轴事件与 A/B/C 代价卡片。

边界如实说明：本仓库建立于 2026-11-14，六个迭代的开发提交未随仓库迁移，因此迭代部分以 prompt-snapshots/ 的 11 张真实对话截图与 AI协同对话历史.md 为证据（赛题允许以截图/导出替代工具内历史）；赛期内的十次改动均有真实提交可 git show 逐条核对，清单见 replay.md 第 3 节。

10. 创新点小结

10.1 与同类产品的本质区别

市面上的个人效能产品可归为三类，但都没有解决「决策代价不可见」的核心问题：

产品类型	核心逻辑	缺失的能力
AI 助手（豆包/ChatGPT）	用户提需求，模型直接给方案	替用户做决定，方案不可复现，完成率低
日历/待办（滴答/Notion）	用户填时间，系统提醒	冲突只标红，不展示金钱/睡眠/后悔代价
习惯养成（小日常/Forest）	打卡记分，断签清零	掉一天就想重开，没有决策视角

表 10-1 同类产品对比

Altiora 的定位不是「更好的助手」，而是把每个选择的代价摊在桌上的决策参谋——不替用户决定，只把得失算清楚。以下六个创新点支撑这一定位。

10.2 六大创新点

- 代价账单代替记分板：冲突不替用户选，而是把 A/B/C 的代价（金钱/时间/睡眠/后悔）摆出来。
- LLM 只做翻译不做计算：确定性排程与 LLM 表达分离，数字只在计算器产生、在表达层校验，可复现且可信。

- 三层意图控制锁：SYSTEM + _BANNED + reply 丢弃/数字校验，配合失败回退，把 LLM 锁死在翻译官角色。
- 档案不是摆设：精力高峰、睡眠硬约束、软肋真实影响排程，改一个参数方案肉眼可见地变。
- 记忆三层 + 久别重逢：长期/聊天/日记用保留时长划边界，3 天以上回访日记，既不监视又不冷漠。
- BYOK + DNS 重绑定防护：Key 只存浏览器，自填 URL 双层校验防内网探测。

10.3 术语边界：我们说的「孪生」是什么、不是什么

赛题名含「数字孪生具身仿真」。为避免把赛道词当成卖点夸大，这里明确本项目使用这个词的范围——它只有一层含义：把用户身上可计算的部分建成模型，供本地确定性算法使用。

约束类别	落成什么参数	影响什么
时间约束	课表、固定日程、缓冲时长	空档搜索与冲突检测
能力约束	精力高峰时段、睡眠硬边界	任务被摆到哪个时段
沟通约束	用户自述的软肋与底线（如「吃软不吃硬」）	回话口吻，且只在匹配时引用原话
节奏约束	目标频率、每次时长、截止日期	计划排到哪一天、排几场

表 10-2 「可计算的你」的四个维度

- 不做的三件事：不生成第二个「你」来对话；不预测你哪天会放弃；不给坚持率打分。前者是拟人仿真，后两者会把陪跑变成审判——既超出单文件前端的能力，也违背「不评判、不追责」的产品原则。
- 架构边界（同样说清楚）：当前是无后端、无账号的单文件前端，数据存浏览器 localStorage，换设备需手动导出导入，也没有推送能力。这不是包装成优点的设计选择，是本次赛期内的工程取舍；代价与替代方案见 7.4 与 11 节。

把边界写出来，比把能力写满更有用：评审可以据此判断哪些是已交付的，哪些只是设想。

11. 部署方案

11.1 零依赖模式（推荐评委使用）

双击 Altiora-未来的我.html 即可。文字对话、排程、冲突代价、记忆、今日方案全部在前端完成，不需要 Python、不需要联网。

11.2 语音模式

环境要求：Python 3.10 及以上（代码使用了 `str | None` / `list[dict]` 等类型注解）。

```
pip install -r requirements.txt
python3 scripts/download_vosk.py    # 仅首次，下载语音模型
python server.py                    # 启动后打开 http://localhost:8000
```

11.3 AI 在线模式

页面右上角点状态标签 → 填自己的 API Key → 保存。Key 只存本机浏览器，请求直连厂商。支持 DeepSeek / 通义 / 火山方舟等 OpenAI 兼容接口。

11.4 评委直接体验：服务端同源部署

除零依赖与 BYOK 两种方式外，评审版另提供一台已配置好的公网服务，评委打开即用，无需任何设置：

```
http://82.157.162.252:8000/        # 打开即「AI 在线」
```

该服务由 server.py 同时托管前端页面与 /api 接口，Key 只存在服务器 .env 中，不下发浏览器、不进仓库。以下为可复现的常驻部署方式（Linux）：

```
# 1) 代码与依赖（在仓库根目录，不要在 Altiora/ 子目录里启动）
cd ~/Altiora.luck
pip3 install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple

# 2) 配置 Key（.env 必须放在仓库根目录）
cp .env.example .env
vi .env      # 填 LLM_API_KEY / LLM_BASE_URL / LLM_MODEL

# 3) 开机自启（systemd），替代 nohup，服务器重启后自动拉起
sudo tee /etc/systemd/system/altiora.service > /dev/null <<EOF
[Unit]
Description=Altiora AI Proxy
After=network.target

[Service]
Type=simple
User=$USER
WorkingDirectory=$HOME/Altiora.luck
ExecStart=$(which python3) server.py
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now altiora
sudo systemctl status altiora     # 看到 active (running)
```

- 启动目录是硬约束：BASE = Path(__file__).resolve().parent 决定它去哪里找 Altiora/backend/llm.py、主页面与 .env。必须在仓库根目录启动、.env 也放根目录，放进 Altiora/ 子目录等于没配。
- 端口：默认 8000，需在云厂商安全组放行入站 TCP 8000；换端口改 PORT 环境变量。
- 日志与重启：日志 ~/Altiora.luck/server.log（nohup 方式）或 journalctl -u altiora -f（systemd 方式）。
- 实测：首页 200、/api/health 返回 llm:true、/api/understand 返回 source:"llm"、日程落地与刷新持久化均通过；单轮耗时约 5 秒（DeepSeek-V3）。

11.5 线上验收记录（2026-11-14）

部署完成后，从异地网络对公网地址做了一次只读验收，命令与结果如下，评委可原样复跑：

```
$ curl -s -o /dev/null -w "%{http_code}" http://82.157.162.252:8000/
200
$ curl -s -o /dev/null -w "%{http_code}" http://82.157.162.252:8000/favicon.ico
204
$ curl -s http://82.157.162.252:8000/api/health
{"ok":true,"llm":true,"model":"deepseek-ai/DeepSeek-V3","asr":false}
$ curl -s http://82.157.162.252:8000/ | md5sum      # 与仓库根目录成品一致
d8f1ed60da18e70f91ed43aa44777ea1  -
$ systemctl is-active altiora && systemctl is-enabled altiora
active
enabled
```

检查项	判据	本次结果
页面可访问	HTTP 200	通过
与仓库成品一致	线上页面 md5 = 仓库 Altiora-未来的我.html 的 md5	通过（d8f1ed60… ）
模型已接通	/api/health 的 llm 为 true，model 为配置值	通过（DeepSeek-V3 ）
无控制台 404	/favicon.ico 返回 204（浏览器不会为缺图标刷红字 ）	通过
重启后自动拉起	systemctl is-active / is-enabled 均为 active / enabled	通过

表 11-1 公网部署验收项

说明：模型对「只说了起点、未说终点」的句子会返回空事件（服务端校验要求起止齐全，宁可丢也不编时长），前端随即用本地规则按 1 小时补全并明说，因此用户侧结果不受影响——这是「宁可问一句也不瞎编」的取舍，不是故障。

12. 不足与展望

12.1 当前不足

- 目标抽取依赖正则锚点词，对复杂句式（如「要是能…就好了」）覆盖有限，未来可引入轻量意图分类模型。
- 代价账单的「后悔值」目前是定性描述，未量化，说服力有限。
- 记忆三层均为本地存储，换设备需手动导出导入，暂无云端同步。
- 单 HTML 文件虽便于部署，但代码量增长后可维护性下降，未来可拆分为模块化工程。

12.2 未来展望

- 引入用户行为反馈（实际完成/未完成），让分身根据真实执行情况动态调整排程策略。
- 代价量化：把睡眠债、后悔值等抽象代价转成可比较的分数，辅助决策。
- 多分身对比：同一目标下生成多种排程风格的分身，用户可对比选择。
- 端到端加密的记忆同步，在保护隐私前提下支持跨设备。

—— 全文完 ——